

Regras Lógicas e Simplificação Booleana

Núcleo Cognitivo da Aurora Siger (NCAS) — Cap 1, Fase 5

1. O que o sistema classifica automaticamente

A cada alerta analisado, o Núcleo Cognitivo faz duas verificações automáticas. A primeira é de autorização: o operador pode consultar aquele módulo? A segunda é de classificação: aquele alerta é crítico e deve subir ao Centro de Comando?

Cada verificação foi modelada como uma expressão booleana e simplificada por um teorema diferente: a primeira por De Morgan, a segunda pela propriedade distributiva. As duas formas de cada regra continuam no código-fonte, e a função `tabela_verdade()` avalia ambas para todas as combinações de entrada, comparando os resultados em tempo de execução.

2. Regra A — Criticidade do alerta

2.1 Enunciado e variáveis

Um alerta é considerado crítico quando há falha detectada em um módulo de prioridade operacional CRÍTICA, ou quando há falha detectada acompanhada de consumo energético elevado.

VARIÁVEL	SIGNIFICADO	ORIGEM DO DADO
FALHA	Há falha detectada no alerta	campo <code>falha_detectada</code> do alerta
MOD_CRITICO	Módulo com prioridade CRÍTICA	<code>prioridade_operacional</code> do módulo
CONSUMO_ALTO	Consumo energético elevado	campo <code>consumo_elevado</code> do alerta

`MOD_CRITICO` não vem do alerta. Vem do cadastro do módulo herdado da Fase 4. A classificação não confia no rótulo de prioridade que o operador digitou: ela cruza a ocorrência com a importância do módulo para a sobrevivência da colônia.

2.2 Expressão original e simplificação

Original:	<code>CRITICO = (FALHA AND MOD_CRITICO) OR (FALHA AND CONSUMO_ALTO)</code>
Distributiva:	<code>CRITICO = FALHA AND (MOD_CRITICO OR CONSUMO_ALTO)</code>

A variável `FALHA` aparece nos dois termos do `OR`. A propriedade distributiva da álgebra booleana permite colocá-la em evidência, do mesmo modo que a fatoração $a \cdot b + a \cdot c = a \cdot (b + c)$. A expressão passa de três operações, dois `AND` e um `OR`, para duas: um `AND` e um `OR`.

2.3 Prova por tabela-verdade

FALHA	MOD_CRIT	CONS_ALTO	ORIGINAL	SIMPLIF.	IGUAIS
False	False	False	False	False	SIM
False	False	True	False	False	SIM

FALHA	MOD_CRIT	CONS_ALTO	ORIGINAL	SIMPLIF.	IGUAIS
False	True	False	False	False	SIM
False	True	True	False	False	SIM
True	False	False	False	False	SIM
True	False	True	True	True	SIM
True	True	False	True	True	SIM
True	True	True	True	True	SIM

As oito combinações produzem resultados idênticos nas duas formas. A linha destacada (FALHA verdadeira, demais falsas) é a que demonstra que a regra realmente discrimina: existe falha e, ainda assim, o alerta não é classificado como crítico. Corresponde ao alerta #3 da base de dados: falha intermitente no Laboratório de Pesquisa, módulo de prioridade MÉDIA, sem consumo elevado.

2.4 Por que a simplificação preserva o resultado

Na forma original, FALHA é condição obrigatória dos dois termos do OR: se FALHA for falsa, ambos os termos são falsos e o resultado é falso, independentemente das demais variáveis. A forma simplificada apenas torna essa obrigatoriedade explícita, colocando FALHA como primeiro fator de um AND. As duas formas dizem a mesma coisa: é preciso haver falha, e além disso pelo menos uma das duas agravantes. A diferença é que a segunda escreve FALHA uma vez em vez de duas.

2.5 Implementação

```
def eh_critico(falha, mod_critico, consumo_alto):
    """Regra de criticidade SIMPLIFICADA (usada pelo sistema)."""
    return falha and (mod_critico or consumo_alto)

def eh_critico_original(falha, mod_critico, consumo_alto):
    """Forma ORIGINAL, so para comparacao."""
    return (falha and mod_critico) or (falha and consumo_alto)
```

Os parênteses em torno do OR são obrigatórios: em Python o operador AND tem precedência maior que o OR, e sem eles a expressão seria avaliada como `(falha and mod_critico) or consumo_alto`. Essa é uma regra diferente, que classificaria como crítico qualquer alerta com consumo elevado, mesmo sem falha alguma.

3. Regra B — Bloqueio de acesso (De Morgan)

3.1 Enunciado e variáveis

O acesso aos dados de um módulo só é liberado se o operador estiver autorizado e o módulo estiver em operação. O bloqueio é, portanto, a negação dessa conjunção.

VARIÁVEL	SIGNIFICADO	ORIGEM DO DADO
AUTORIZADO	Operador possui autorização	informado no menu do sistema
MODULO_ATIVO	Módulo em operação	status == "OPERACIONAL" do módulo

3.2 Expressão original e simplificação

Original: `BLOQUEAR = NOT (AUTORIZADO AND MODULO_ATIVO)`

De Morgan: `BLOQUEAR = (NOT AUTORIZADO) OR (NOT MODULO_ATIVO)`

O **Teorema de De Morgan** estabelece que a negação de uma conjunção equivale à disjunção das negações: $\text{NOT}(A \text{ AND } B) = (\text{NOT } A) \text{ OR } (\text{NOT } B)$. Aplicado à regra, ele transforma uma negação global em duas negações locais ligadas por OR.

3.3 Prova por tabela-verdade

AUTORIZADO	MODULO_ATIVO	ORIGINAL	SIMPLIF.	IGUAIS
False	False	True	True	SIM
False	True	True	True	SIM
True	False	True	True	SIM
True	True	False	False	SIM

3.4 Por que a simplificação preserva o resultado, e o que ela acrescenta

As quatro combinações coincidem: o bloqueio ocorre em três dos quatro casos, e só não ocorre quando as duas condições são simultaneamente verdadeiras. Isso é esperado, já que um AND só é verdadeiro quando ambos os operandos o são. Sua negação, portanto, é falsa apenas nesse único caso.

O ganho aqui é de outra natureza. Não houve redução de operações, já que as duas formas usam duas. O que muda é a informação disponível: a forma original produz um veredito único, sem causa, enquanto a de De Morgan mantém as duas causas escritas separadamente. Com elas separadas, o sistema testa cada uma e informa ao operador o que impediu o acesso:

```
if bloquear_acesso(autorizado, modulo_ativo):
    if not autorizado:
        print("ACESSO NEGADO: usuario sem autorizacao.")
    else:
        print(f"ACESSO NEGADO: o modulo {alerta['modulo']} esta INATIVO.")
```

Em execução, essa distinção é visível: analisar o alerta #6 (Campo Solar Sul, único módulo INATIVO da colônia) produz a mensagem sobre o módulo; analisar o alerta #1 com autorização negada produz a mensagem sobre o operador. Nenhuma das duas seria possível com a expressão original.

3.5 Implementação

```
def bloquear_acesso(autorizado, modulo_ativo):  
    """Regra de bloqueio SIMPLIFICADA por De Morgan (usada pelo sistema)."""  
    return (not autorizado) or (not modulo_ativo)  
  
def bloquear_acesso_original(autorizado, modulo_ativo):  
    """Forma ORIGINAL, so para comparacao."""  
    return not (autorizado and modulo_ativo)
```

4. Aplicação integrada no sistema

As duas regras são aplicadas em sequência pela função `analisar_alerta()`, acionada pela opção 3 do menu. A ordem é deliberada. A Regra B é avaliada primeiro porque trata de autorização: com o acesso bloqueado, o sistema não deve calcular a criticidade nem exibir o conteúdo do alerta a quem não pode consultá-lo. Só depois de liberado o acesso a Regra A entra e a classificação é produzida.

- **Redução de operações.** A Regra A passou de três operações lógicas para duas.
- **Ganho de expressividade.** A Regra B passou a distinguir as duas causas de bloqueio.
- **Equivalência verificável.** A opção 4 do menu reexecuta as doze linhas de prova a qualquer momento.

A classificação automatizada é tratada como apoio, e não como veredito. Ao final de cada análise o sistema imprime que a decisão final é responsabilidade da equipe humana do Centro de Comando.